

Sheffield Hallam University
Faculty of Arts, Computing, Engineering and Sciences (ACES)
Department of Computing

MComp Individual Project Report

Can a Compute Shader Octree Approach to Render Culling in Vulkan Increase Performance Compared to Traditional CPU-GPU Approach.

By Harrison G. Henley

28.04.2025

Supervised by: Mark Featherstone

This project report does NOT contain confidential material and thus can be made available to staff and students via the library.

A report submitted in partial fulfilment of the requirements of Sheffield Hallam University for the degree of Master of Computing in MComp Computer Science for Games

Introduction & Overview

In graphic rendering, there is a demand from consumers for better performance/ higher-level efficiency of high-quality graphics, as such, these consumers buy new hardware & consistently update the graphic drivers to satiate that need.

And behind the scenes, programmers & engineers are spending time to enhance the performance or efficiency of rendering, either on hardware, creating new graphics cards, or software, utilising rendering SDKs (Software Development Kit) to develop inventive directions for graphics rendering to go.

This paper will be employing a rendering SDK named Vulkan, this is because of its versatility in graphics programming and allowing the user to personally construct graphic pipeline(s) suited for the appropriate purpose.

In addition, Vulkan allows the use of non-graphics-based tools, such as compute shaders, which this paper will be exploiting.

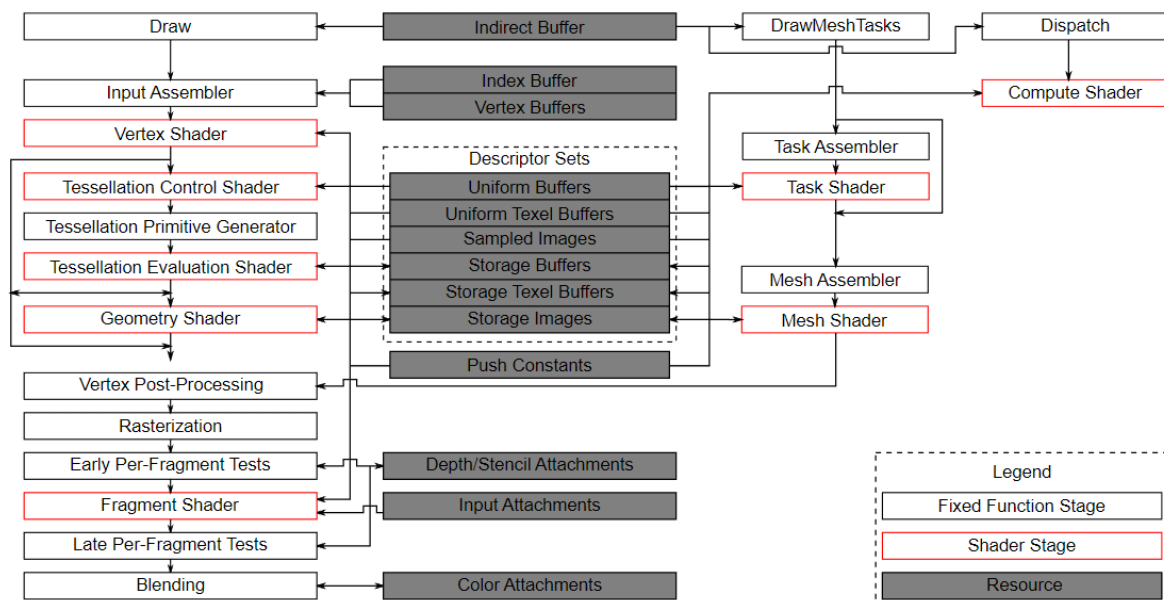


Figure 1: Compute Shader usage inside of a graphics pipeline.
(Vulkan, n.d.)

Other SDKs such as: DirectX11, SFML or OpenGL, could not be considered as they did not allow for such engine building that this paper is

requiring. And all SDKs like Vulkan, e.g. DirectX12, were considered but ultimately dropped for preference of Vulkan's lower CPU overhead.

The goal for this paper is more efficient / better performance of high-level graphics, resulting in an appropriate SDK and knowledge about compute shaders, leaving only now a direction to utilise these tools.

One such direction could be binary space partitioning; an older deprecated technique used in early graphics to cull the models / images outside the view of the camera.

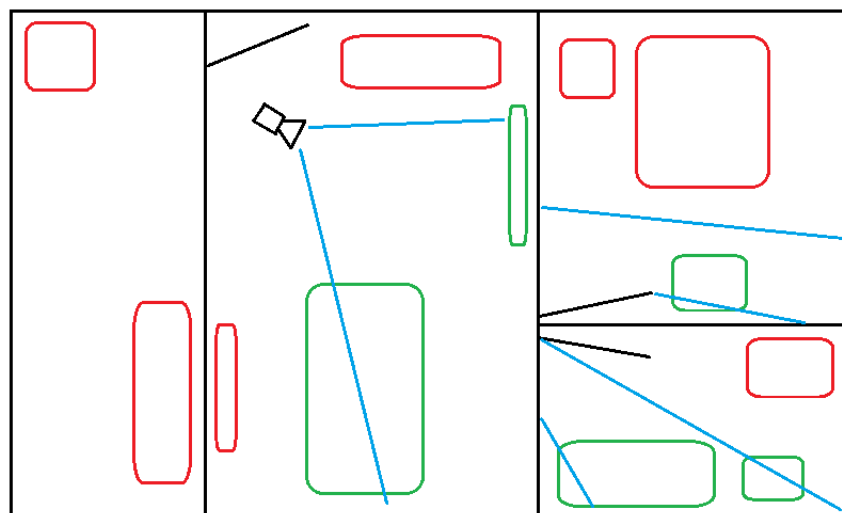


Figure 2: Example of Binary Space Partitioning within a house, blue lines represent the camera's bounds, green boxes – models rendered, red boxes – models not rendered.

Shown in Figure 2, in the bottom-right most room, a red box lies in the corner of the room, to demonstrate the culling that takes place, and this object could be millions of polygons, significantly reducing performance.

Although sounding like a great technique, this process is cost expensive and can cost the end-user performance drops and if the scene was dynamic, it becomes inefficient, having to be ran as many times as possible in as quick of a time possible.

Therefore, the Binary Space Partitioning rendering technique is quite lacklustre, when attempting to render a dynamic scene. On the other hand, it can be said that many methods bend under stress and that loss in performance is required for the change to dynamic objects.

The reason that Binary Space Partitioning is depreciated is due to its limitations, some of these are: this technique must be done on a plane or 2D representation of the scene, as shown in Figure 2, and for a 3D render, multiple planes could be required.

Another limitation is the numerous times the GPU would have to cut up the scene into different areas and lines. This in combination with the previous limitation shows the stress the GPU would be under rendering a dynamic scene.

Additionally, if an object were to clip into another overlapping the models, there could be unpredictable behaviour from the renderer.

In summary, the Binary Space Partitioning method isn't feasible to be used, going against the demand from consumers and seems depreciated in comparison to other rendering methods.

Thus far, the rendering method that should be implemented for this paper should satisfy a minimum 3 requirements:

1. It should be capable of being used within a compute shader, to have more efficient rendering done only on the GPU, compared to the standard CPU-GPU method.
2. The method should show the distinguishable difference between itself and the standard rendering method.
3. The chosen technique must be able to be used in a 3D scene.

With these conditions in mind, the next step would be to do 3D Spatial Partitioning, very much alike to Binary Spatial Partitioning, with the noticeable difference being that the former separates 3D space to form boundaries used for collision and/or rendering.

The next step is to determine the way the space should be partitioned, different methods would provide different results, one such way, is Bounding Volume Hierarchy.

This method uses a per object system, giving all models a uniformed grid

that forms around the outline of a model and broader faced collision boxes generated as well.

This method is usually used for narrow-face collision detection and is quite good for rendering per object. Although, there are a few issues with this method, mainly that this would go through all objects within a scene, creating different boxes / grids around the models and within a large scene, would impact performance significantly. Another problem is that, due to the nature of Bounding Volume Hierarchies, it would be unfitting to the 4th requirement, not being able to be utilised with the compute shader.

Yet the idea of using a uniformed grid, instead generated around an entire scene, allowing the level of detail for further away areas to be lowered or left unrendered.

So, the technique that meets all the criteria, would be 3D spatial partitioning using an octree approach, processed with the assistance of a compute shader.

Literature Review

Painter's Algorithm

A small bit of history, back in the early 1960s, when computer graphics was just starting out, all graphics was done using the “painter’s algorithm” (GeeksforGeeks, 2020), this algorithm was the rendering of a scene from back to front, by sorting the polygons within the image by their depth.

This algorithm had some obvious issues with it, for starters, instead of rendering pixel by pixel (of a screen), it rendered polygon to polygon by the sorted container, which is a significant slowdown in processing speed, especially if there were millions / billions / trillions of polygons in comparison to a set number of pixels.

And graphic pipelines using this algorithm, for a dynamic scene, would be running innumerable number of times, slowing any CPU to a halt, to constantly try and keep up with the dynamic objects within a scene.

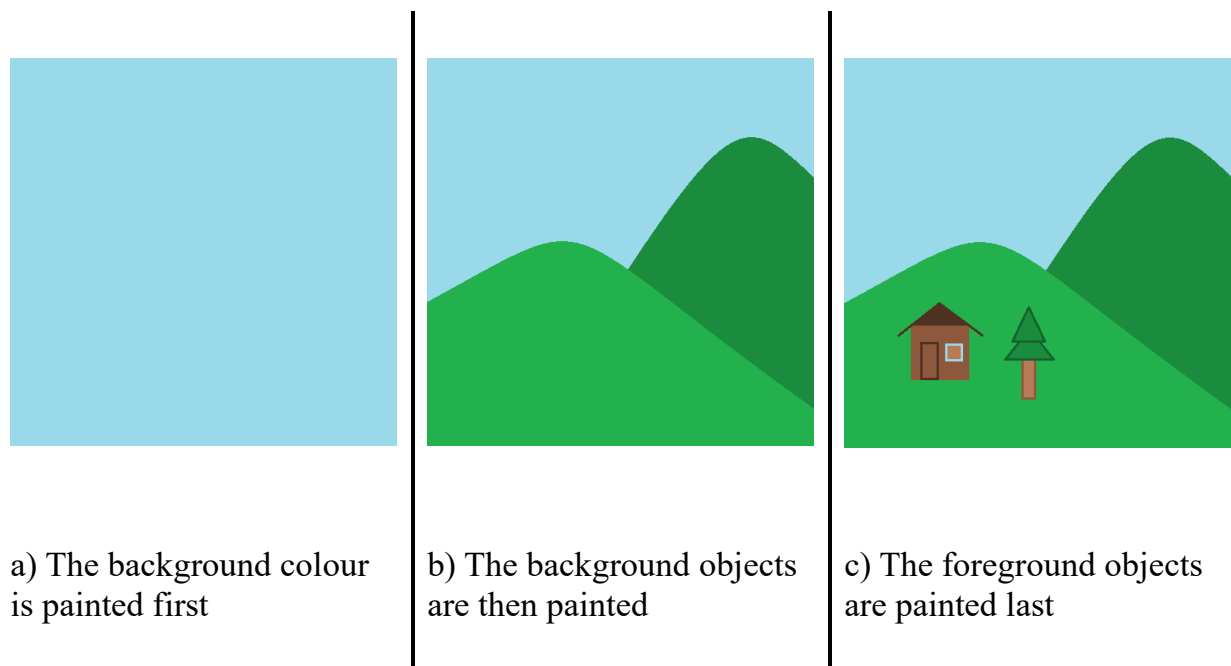


Figure 3: Painter's Algorithm Process.

Then, in the late 1960s, the “study for applying computer-generated images to visual simulation” was published (Study for Applying

Computer-generated Images to Visual Simulation, 1969), this was used to assist in rendering: windows, liquids, or any translucent object. This algorithm makes use of depth coherence, and states that a polygon of the far side of a plane, cannot obstruct a closer polygon, later named Binary Space Partitioning.

However, this method needed to have the polygon data organised, and this task needed to be performed manually by the scene designer. This significantly slowed down the speed at which rendering could happen and make the overall process more cumbersome and tiring for the designer(s).

Binary Spatial Partitioning

And over the 1980s / 90s, the method was improved upon: representing 3D objects in a virtual environment by using planes that intersect with polygons to recursively partition 3D space, development of Binary Space Partitioning Trees & a graph-theoretic approach for pre-computing visibility, examples of real-time visible surface determination, etc.

Amidst these developments, Doom (1993) was released using the Binary Space Partitioning method as a core part of its engine, for optimising both the rendering and collision detection during real-time.



Figure 4: Doom, 1993, Binary Spatial Partitioning of the 1st Level.
(DoomWiki.org, n.d.)

Doom (1993) showed huge financial success and stands as one of the most important steps in improved rendering. As seen in Figure 4, there are numerous partitions that have been created by the designer, done to remove or reduce the time taken for a frame to be rendered.

Which shows the benefits of moving away from the "painter's algorithm" way of rendering and transitioning into front-to-back rendering, such as high frame rates, low loading times and memory efficiency.

However, this method of rendering would not last forever, due to consumers wanting improvements within their games, like larger levels or higher quality models.

And with Binary Spatial Partitioning needing to be manually carved out into partitions by a designer, it makes sense that a new method would be required.

A large turning point in transitioning from binary spatial partitioning is the release of NVIDIA's GeForce 3, a GPU (Graphics Processing Unit) allowing users to perform low-level manipulation of the shaders.

Rasterization

This GPU allowed rasterization to be refined, improving its efficiency and quickness, and compared to spatial partitioning, rasterisation could run through the graphics pipeline far faster. And another major improvement, was that there was no designer required to layout the scene before-hand.

Rasterisation is the rendering technique, where the vertex shader populates the scene with objects made of polygons, but rather than culling non-visible objects from the scene, an image inside the vertex shader is created with the polygons visible only to the camera.

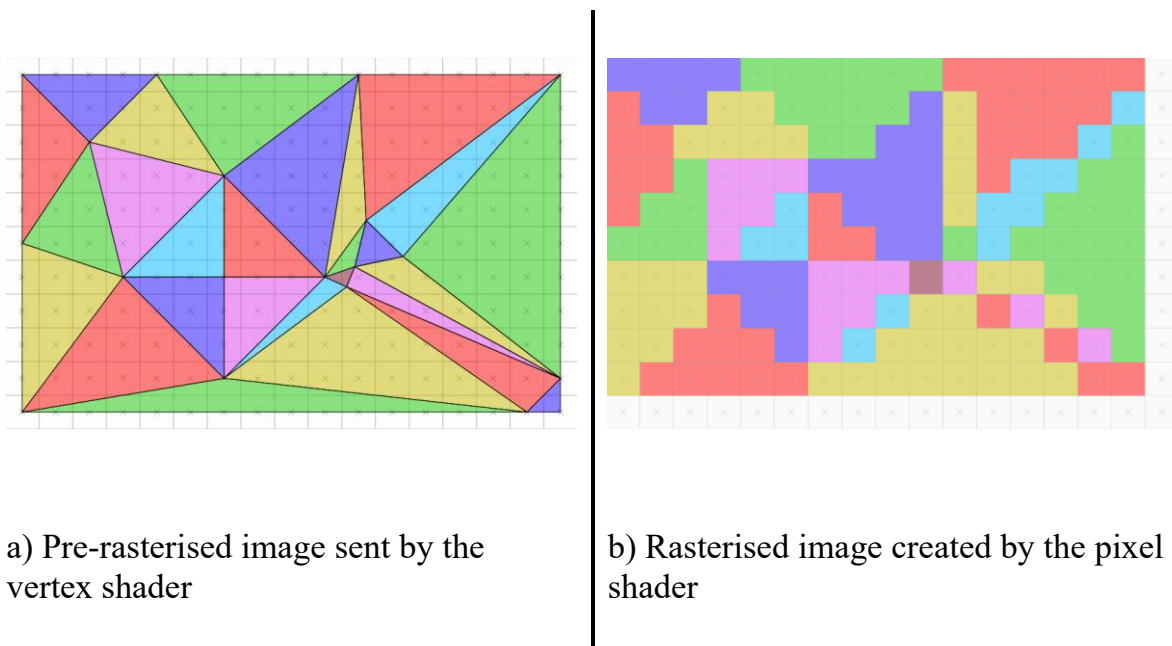


Figure 5: The process of rasterisation (Drummyfish, Wikipedia)

And in the pixel shader, this image is then used to create a new image, comprised of squares of solid colour, this technique can be shown in Figure 5.

This technique is further improved upon when anti-aliasing is used in tandem, creating fewer sharp edges and more smooth corners.

However, rasterisation suffered from having to do this process many times over thousands of pixels and with no way for a developer to alter the shaders, the technique was outdone by binary spatial partitioning.

So, when NVIDIA released the GeForce 3, this issue with raster graphics became negligible and with improvements to the shaders in the next few years, the quality of graphics rose significantly, showing vastly different results.



a) Halo: Combat Evolved, using binary spatial partitioning (Retro Review – Halo: Combat Evolved – Here We Go., 2017)



b) Halo 2, using rasterisation (Probably a hot take but does anyone else think Halo 2 classic graphics are pretty bad for 2004., 2023)

Figure 6: Halo, 3 Years Apart, 2 Different Rendering Techniques

Figure 6 shows how low-level manipulation of the shaders, as far back as the early 2000s, Figure 6b displays much higher quality in the textures and models, and graphics such as these caused the demand for better graphics to sharply rise.

And with this demand, meant better hardware was required for both the end-user / consumer, and the programmers needing to code the software suitable for these games.

Then, from the early 2000s to 2018, rasterisation was the main go-to for graphics pipeline, with other attempts of different techniques being used to try and discover better ways of improving graphics, all the while hardware was being improved upon through the years.

Raytracing

Until 2018, when NVIDIA released the RTX 20 Series, this GPU supports raytracing so that it can be utilised in real-time, which supports the push consumers want for graphic improvements.

In modern times, raytracing is the most used rendering technique within games, for its multiple uses and greater depth for detail.

It does this by casting a ray on every pixel on a screen and returning either hits or misses; hits would return the colour of an object within a scene, while a miss would return a plain background colour. (Shirley et al., 2025)

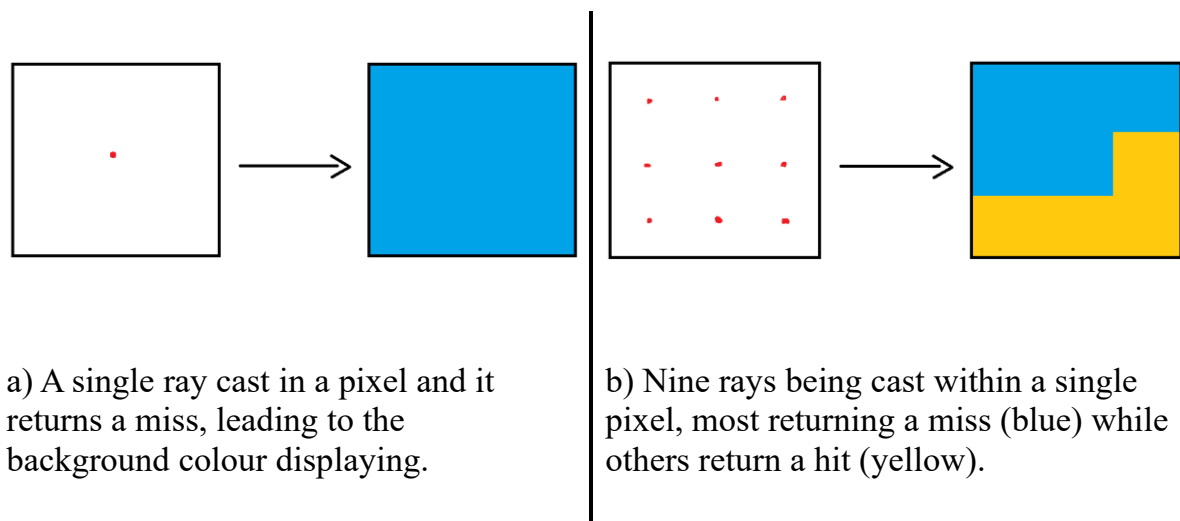


Figure 7: Raytracing within a pixel visual representation

This is then expanded upon, adding more rays within a pixel, resulting in an image with greater detail per pixel. However, for each ray cast, more processing power is needed, and for games, this process would have to happen in a dynamic scene costing the consumer time and frames.

On the other hand, this technique sounds unequal for consumers with worse PCs, raytracing is unlikely to be utilised for generating a whole game's visuals, but it thrives on improving lighting within a scene, likely due to how similar the concept of raytracing is to real world light rays.

This technique is done the same way mentioned previously, Figure 7, but by changing how the rays traverse the scene, the lighting within a scene wildly changes.

Raytracing for light sources only change the rays' destination, changing from hits and misses to distance between where the ray initially hits and the light source. Resulting in, depending on the number of samples per pixel, highly detailed shadows, that is more efficient than shadow maps.

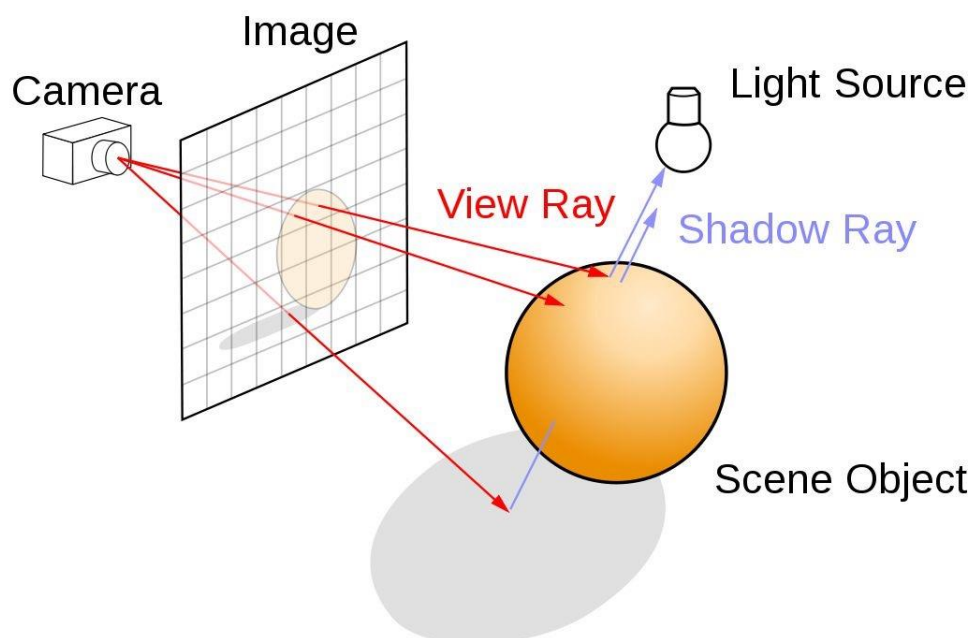


Figure 8: NVIDIA's Developer Blog on Raytracing showcasing the process of raytracing within a diagram. (NVIDIA, n.d.)

Furthermore, this technique is improved upon by having the ray 'bounce' more than once, each bounce sending more data back to the renderer adding realistic lighting. This method is called path tracing evolving from raytracing.

But enough of the lighting rendering, what other vertex loading methods are there?

Voxelisation

There is another method where the world is generated via cubes called ‘voxels’, these voxels are an efficient way of both: rendering entire scenes and/or aid in lighting calculations.

When rendering entire scenes, the original mesh is input into the shader for ‘voxelisation’, allowing the mesh to transform into a voxelated version of itself. And is mainly used for the level of detail within games, greatly reducing rendering time for every frame.

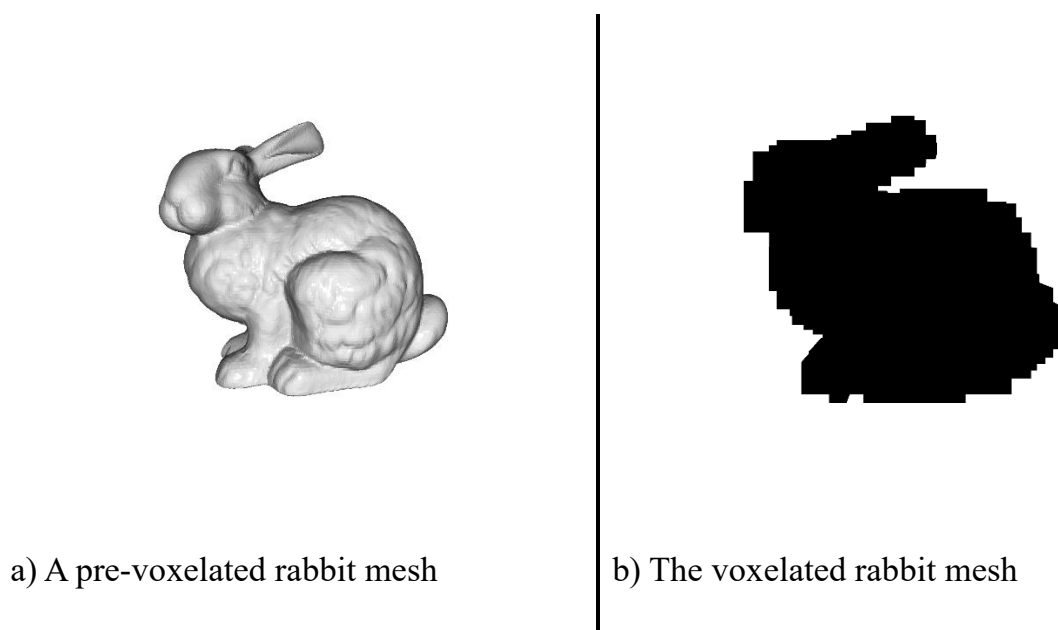


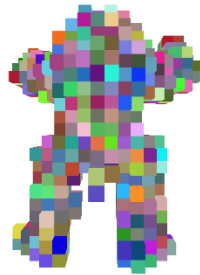
Figure 9: open3D’s Voxelisation Tutorial Example of Mesh to Voxels (Open3D, 2020)

Figure 9 shows this effect of voxelisation, with the rabbit mesh in Figure 9a transforming into the voxelated mesh in Figure 9b.

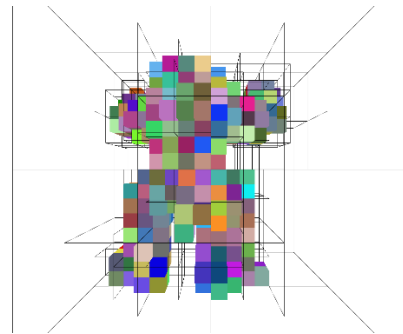
At further distances, the voxelated mesh would seem no different from the non-voxelated mesh, the only difference would be the renderer taking more time calculating the vertices of the non-voxelated mesh rather than the other.

However, the main disadvantage of voxelisation is the creation and recreation of these voxelated meshes. So, a method used to assist with voxelisation is octrees.

Octrees are the division(s) of an area that surrounds either: the object or scene, creating a grid that can be searched through, that specify where processes should be done or, for voxels, removing stray voxels generated from clipping.



a) A voxelated mesh



b) An octree grid generated surrounding the mesh, and the removal of anomalous voxels

Figure 10: open3D's Octree Tutorial Example of Octree Grid Culling Stray Voxels (Open3D, 2023)

Such processes can be seen in Figure 10, with the number of voxels being limited, therefore reducing the time required for the renderer to process these voxels, with more complex the object, the more anomalous voxels are generated, and with an octree grid, less voxels need to be processed, the less time spent on each frame.

Now with this in mind, octrees are generated in the vertex shader, shown in Figure 1, along with the voxelisation of the mesh and even the base vertex transformation.

And seeing how octree grids reduce the time taken to render models, but they themselves take time to process, there should be a way of reducing the time taken to create the octrees.

Compute Shader

In Figure 1, after the vertex shader is done, the compute shader may be used to process any post processing needed by the renderer, however, this can be done in the opposite way and instead be used prior to the vertex shader running.

A compute shader is a shader used solely by the GPU to process the same mathematical process, N number of times, for some vertex post-processing techniques.

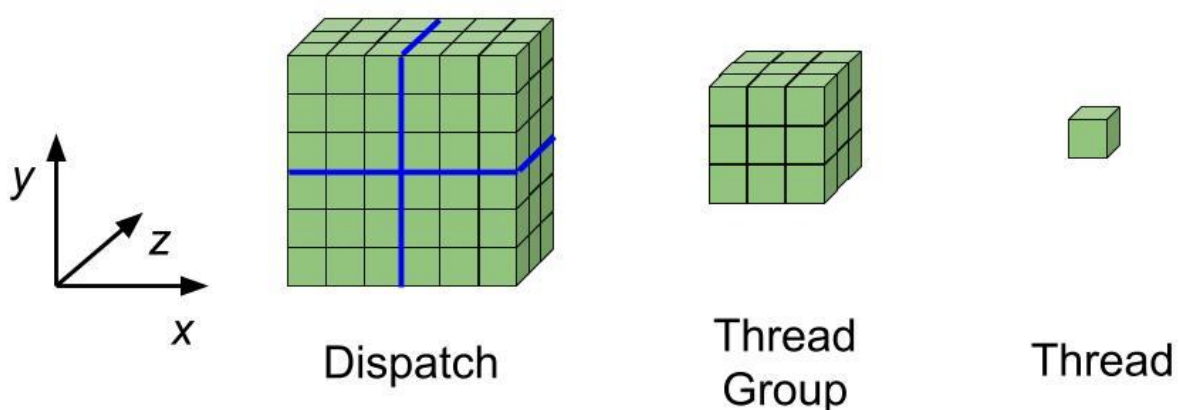


Figure 11: Compute Shader Thread Groups Visual (Loggini, 2022)

Shown in Figure 11 is the process a compute shader takes: starting from the dispatch, where it divides the workload amongst the number of thread groups, that thread group each give a thread, a portion of work to do.

From here, it is very important to note that one of the biggest strengths that compute shaders have, when the renderer dispatches some work for the compute shader to process, it directly sends this work to be done to the GPU, and it will allow multiple cores to do the work simultaneously. Therefore, reducing time spent on every frame significantly.

Frustrum Culling Methods

Normally, by implementing render culling to remove vertices to decrease frame time significantly, this is due to how the vertices are determined to be culled. For this there are several different methods as seen in Figure 12:

- View-Frustum, where vertices outside of the view of the camera / eye are removed
- Back-Face, vertices behind the visible vertices of an object are removed
- Occlusion, entire models being removed as they are behind another model.

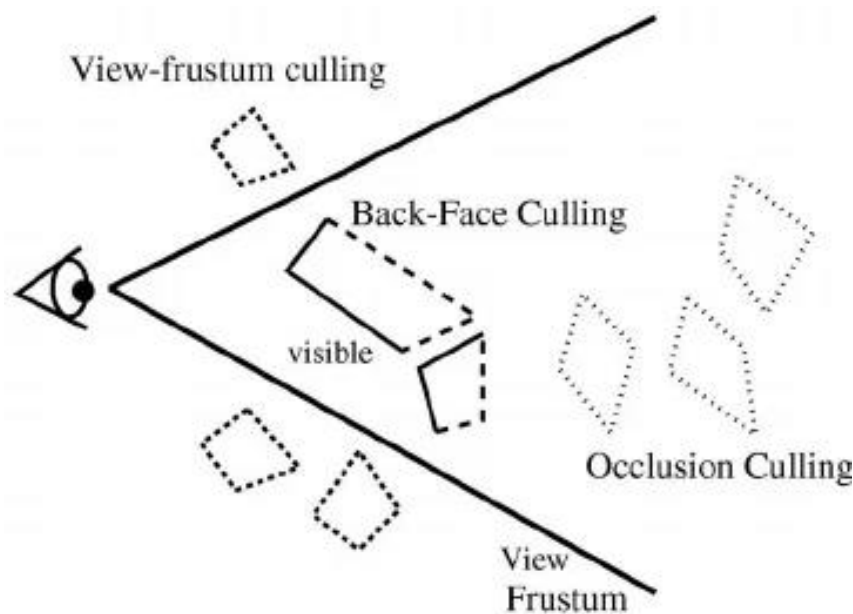


Figure 12: Different Versions of Culling used in Renderers (Chrysanthou, 2001)

Methods & Methodologies

What are the requirements for this project?

In this project, large amounts of data must be processed, used to create the grid in the compute shader, process the vertices passed into the vertex shader, and colour the models within the fragment shader.

The reason that the creation of the grid is done within compute shaders fall to three important factors:

1. The grid must be created prior to any vertex shading meant to take place, so creating it within the compute shader then passing the data over is much easier than creating an entirely new pipeline just to

generate this grid.

2. Creating the grid on the CPU side / on visual studio, then passing it to the vertex shader takes away from what the goal of the project is trying to achieve.
3. If the vertex shader was to be creating the grid, then it would need to be processing huge loads of data that when done in a forward renderer, it could significantly increase frame time.

As shown previously in Figure 11, the dispatch of the compute shader is the number of thread groups that are going to process that data, with the GPU making all its cores dedicated to the compute shader's code.

Not only that, when the thread groups are split into their threads, each thread will run concurrent to multiple other threads, which is called a wave.

Therefore, each wave is the processing of many threads simultaneously, allowing the creation of the grid much faster than if it was processed within the compute shader.

If the compute shader was given the data prior to its execution, for example the minimum and maximum bound(s) of the scene, then the processing of that data would be even that much faster.

Another method that could be used is creating an octree grid surrounding the objects within the scene and checking these grids to precisely cull the vertices that belong to areas that aren't wholly within the frustum of the camera.

However, this change would come at a cost, having to create a detailed enough grid would be simple enough to process, only ever needing to be done once for each model.

But having to find thousands of areas to be within the frustum within the vertex shader could be expensive on both the GPU and CPU sides of the renderer.

On the other hand, precise vertex culling would be more intensive for the GPU and require more time per frame.

An issue with this method, would be dividing the grid for each object, as there could be multiple of the same model inside a scene, however scaled differently, leading to the designer of a scene having to specify the divisions of each object.

What about other low-level APIs?

Other APIs like Vulkan, such as DirectX12, would be a suitable alternative, fulfilling all the requirements for compute shaders and model loading.

However, with the headstart made prior to the start of this project, utilising the online Vulkan tutorial, it was deemed better to have been better to start with the fullest freedom of engine manipulation.

Rather than starter code found online with some of the shaders done for the development already present and would have to be changed.

Any other APIs such as DirectX11 or OpenGL wouldn't have the low-level engine manipulation wanted to achieve the aim.

Are there other graphic techniques that could improve the method?

There are other ways to achieve something like this:

1. Voxels
2. Bounding Volume Hierarchy (BVH)
3. Closest Vertex Usage

Voxels could be used to generate the whole scene in a low precision grid, then using those generated voxels, the technique would stay the same.

However, this voxelisation that takes place would have to be thoroughly and thoughtfully sized as the scene would change.

Not even to mention the amount of processing time that would be required to store and calculate the voxels.

BVH is already quite similar to what is being tested just instead of the whole scene, the vertex shader would test these bounds and remove the object.

But a large drawback of this method is the BVH model itself, generating it through the CPU for many different models would take too much time.

However, since that process only needs to happen once per execution, it would be better in a dynamic setting, especially for larger objects in the scene.

And finally, testing the closest vertex and seeing if that is within that camera's frustum would be fast, however very inaccurate and prone to errors.

For example, if the camera was close to a plane that had a texture on, using the closest vertex wouldn't work and likely just show another object on the other side.

As seen in Figure 13: the frustum lies within the plane and none of the vertices are inside that area, making the plane not visible.

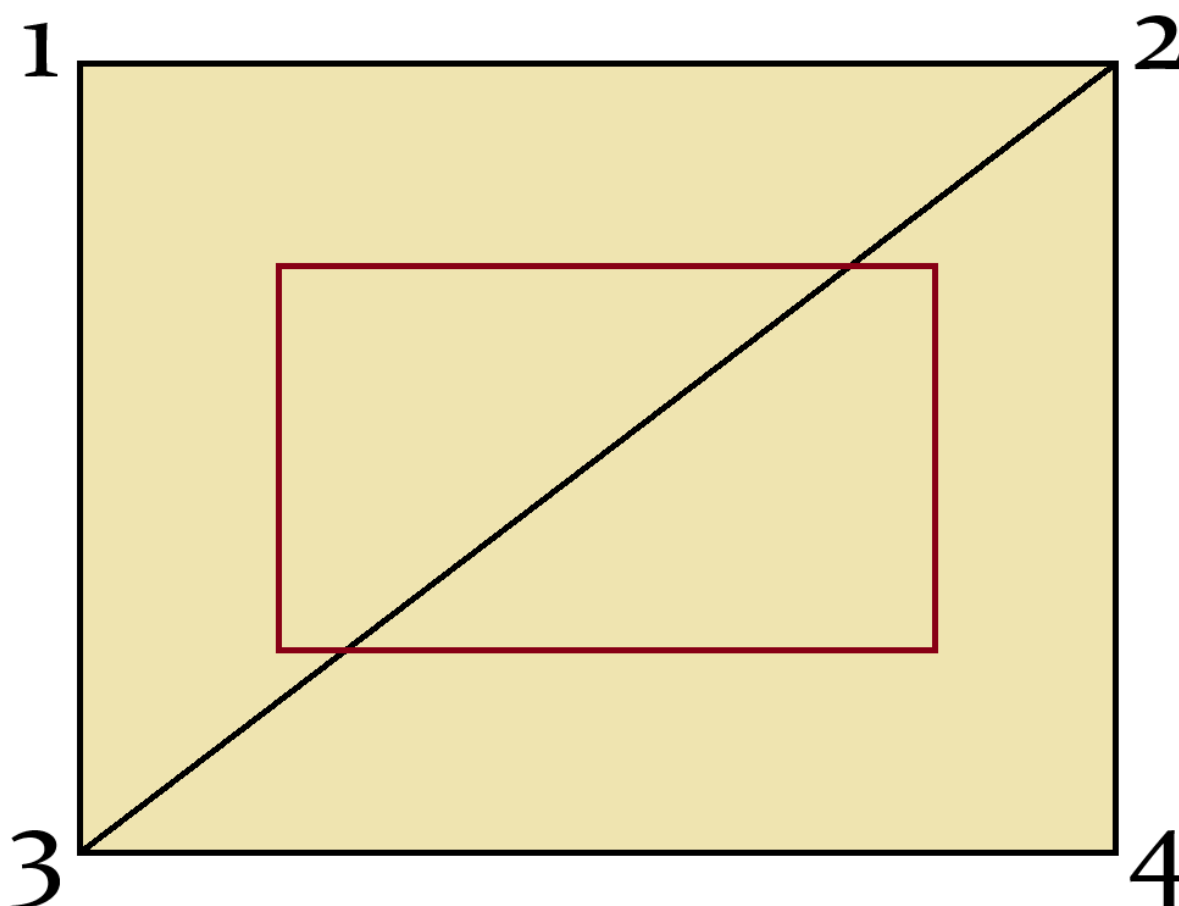


Figure 13: Closest Vertex Example Image, Black Square is the Plane, Red Square is the Camera's View

To conclude these different alternatives, the voxels and bounding volume hierarchies are quite good to test this method with.

That cannot be said for the closest vertex usage, for it being so inefficient and having too many cons that out-weigh the pros.

As well as the fact that if an object's closest vertex is outside of the frustum, but the rest of the object is within the frustum, that object would be cut out.

Is there another way of making the method more efficient?

Without a doubt, there are many ways to develop this technique and raise it to a higher level of efficiency, for example:

- Reducing the memory consumption that the method requires, would be the most likely method of increasing efficiency.
- Reducing the layout of the compute shader or the dispatch command for the compute shader on the CPU side. Therefore, reducing the number of cubes created.
- Adding the compute shader into the normal graphics pipeline and putting the dispatch prior to the execution of the vertex shader in the command buffer.

When adding the compute shader into the graphics pipeline, it should be noted that instead of creating an entirely new pipeline, it would be more difficult to pass the data generated into the vertex shader.

This is because the buffer that stores the vector data pertaining to the different voxels is stored within a buffer, with memory allocated for its size.

Whereas if it was inside the graphics pipeline, the pipeline could go over the amount memory capacity to pass through the data.

Is there a way of implementing this method for a dynamic scene?

When it comes to dynamic scenes, it would be tedious to have the octree cube perfectly align with the scene while having moving objects.

So, to resolve this issue, the entire scene's octree grid would have to be scaled up an appropriate amount to fit as much as possible that even dynamic objects would still be in the scene.

However, this begins to start causing problems:

1. The size of the cubes could lead to problems stated before, as seen in Figure 13.

This can be easily resolved by increasing the layout / command dispatch used by the compute shader, but this only leads to the

second issue:

2. With so many cubes' vertices being stored into memory, the allocation of this memory will lead to Vulkan crashes.

And with these two in mind, a possible solution to this would be to have the octree grid become dynamic also.

Meaning that for every object(s) model matrix, a new octree grid would be formed, which leads to the BVH approach as stated previously.

Yet, the whole scene would not be dynamic, so doing this for the dynamic objects would result in the right solution, the static objects could still use this octree grid for other purposes, such as:

- Lighting
- Shadow Maps
- & Many More...

Design & Development

The Drawn-up Design

Beginning with the initialisation steps needing to take place, the first steps, as shown in Figure 14, start with the model and texture loading.

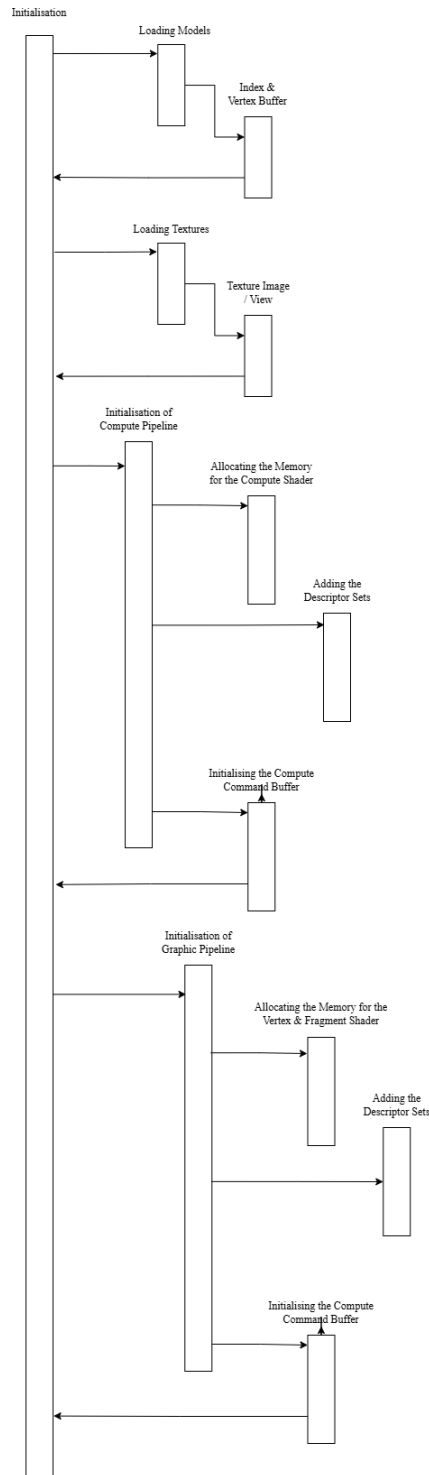


Figure 14: Initialisation Sequence UML Diagram

Model loading takes the data within the files and stores it directly into 2 storage buffers, the index & vertex buffer.

Texture loading also takes the image file and stores it by allocating memory directly into the heap and into an image / view.

However, during the model loading step, a comparison of the vertex's positions to find the highest and lowest valued positions along the 3 axes (x, y, z).

This would create the largest cube around the scene, preparing for the compute shader stage that is implemented in the pipeline next step.

The compute pipeline initialisation stage has 3 main steps:

1. Allocating memory for the compute shader, this will contain all the octree data needed for the comparison.
2. Allocating the descriptor sets, what data the shader should be getting from outside, or what data it is being sent by a different shader.
3. Initialising the compute's command buffer, what data is being changed and contains the command dispatch for the shader.

After that, the graphics pipeline is initialised with the same 3 steps, but there needs to be a 3rd descriptor set within the pipeline to take in the compute shader data.

Thus, allowing the vertex shader to compare every vertex passed in to check whether it is within any of the octree areas.

Now the design was there for the compute pipeline to be implemented, the next step was choosing an API to use.

When the compute pipeline would be implemented the graphics pipeline would look like Figure 15.

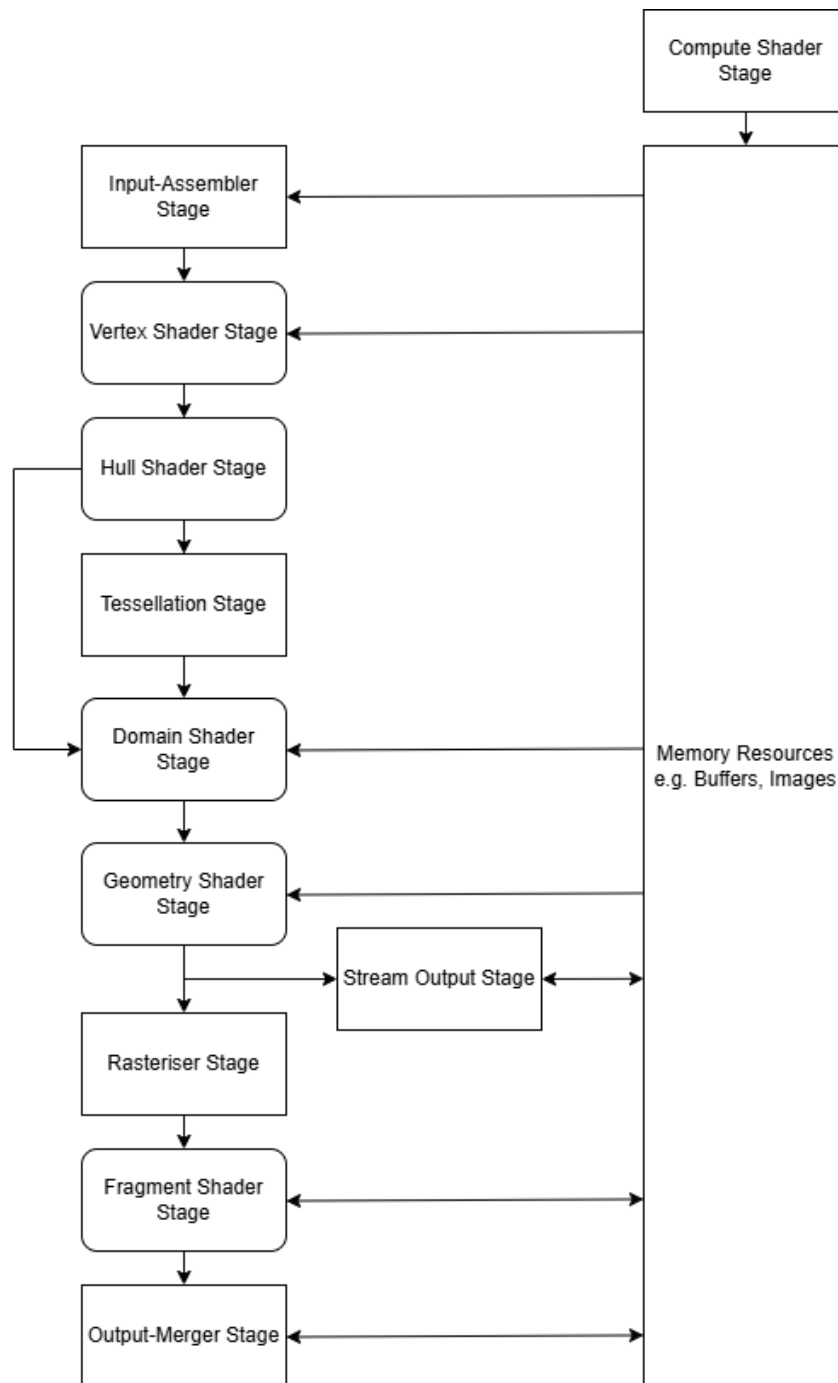


Figure 15: A graphics pipeline with the compute shader stage implemented.

Design Considerations

Starting development on the artefact required both a suitable API, allowing for manipulation of compute shaders, and for it to be something that improves upon performance, showing an improvement in effective memory usage / frames per second within a scene.

Vulkan was chosen as this API due to its flexibility, due to its flexibility and online examples of other graphic techniques.

Raytracing was originally chosen for its frequency in graphics literature and its ability to have more flexibility with post-processing.

This would've met all the requirements the project desired and perhaps, even pushed what the outcome of this project could've been.

Although due to time constraints and for the benefit of the project, raytracing was moved away from.

Then came compute shaders, that could allow easier manipulation of data in the pixel shader and post-processing.

But how could this project utilise compute shaders for graphics to improve memory usage, this is by pairing this shader with voxelisation.

The entire scene could be transferred to octree voxel data and after being transferred into the graphics rendering pipeline.

And then within the vertex shader, it could check every area the vertices belong to and if none of the vertices of the area that model belongs to are within the frustrum, the entire model would be skipped.

But this design has some major flaws:

1. If a model is larger than the area specified then when the vertex shader runs, and the model overlaps into different areas the vertex shader may run into a bug where it can't find the area.
2. The multiple models being loaded must be checked CPU-side to ensure that the correct starting area can be found.
3. Because such a precise cube is created from the vertex data of the model (and the positions of those vertices), there is a chance that some vertices would go out of bounds.

Reviewing these potential issues, there were a limited ways of removing these potential problems before they occur.

For the 1st issue, an easy change that was made was to switch from model removal to vertex removal, checking if vertex 'n' was inside any of these cubes and render

Milestones / Progression

Originally, the NVIDIA Vulkan KHR raytracing tutorial was followed to act as the base of the project, allowing for even greater range in the development of this project.

However, this later changed to the online Vulkan tutorial for a simple graphics pipeline starting point, this was due to the early changing nature this project had.

A great advantage of this change is having greater understanding of what occurs within the renderer and being able to implement compute shaders easier.

What were the Milestones needed to complete the project?

In a concise list of steps:

1. Implement multiple models.
2. Add a compute pipeline.
3. Implement the storage buffer for the compute pipeline's output.
4. Connect the compute pipeline to the graphics pipeline.
5. Create the compute shader.
6. Implement the data into the vertex shader.
7. Test for different output.

Milestone Explanations

The first milestone was to get this project working to implementing multiple models to populate the scene for testing.

There were 2 methods for achieving this:

1. Implementing an object and model class; the object class containing the position, rotation, scale, and the buffers for every object, then the model class being a container for the index, vertex buffer(s), and the texture image.
2. Simply loop through the modelling function and have a bunch of the same model populate the scene.

While option 1 gives a large range of testing options that can be done, such as different scaled objects or complex 3D shapes.

Option 2, for the purposes of the project, is easier to implement and has the added benefit of being able to easily make changes to a single object then go through individual objects.

With these options in mind, and their respective pros and cons, it was found to be better to choose the quicker 2nd option rather than the 1st option.

The main contributor to this decision was the amount of time needing to be spent debugging and testing, as this project's aim is to test whether it is possible to utilise these octrees not to find whether it can be made efficient or not.

Although implementing this took little time, the next milestone would be different and much more complex, for it was to implement the compute shader into a pipeline within Vulkan.

This is a much more tedious task then mentioned previously, and the development ran into some issues:

- The descriptor sets were inconsistent with the pipeline, recording that the pipeline had been given a different layout from that set.
- The uniform buffer had apparently not been passed through to the compute shader.

These errors were consistently being output together with a crash soon to follow, only through removing the terminate function would the errors be able to display on screen.

It turns out that the descriptor set was not having the correct elements bound to it through the attachment descriptions, which was an easy but irritating fix.

The process of adding the uniform buffer for the compute pipeline took only 5-10 hours to resolve, and with that was to progress creating the compute pipeline.

As well as calculating the amount of storage the buffers would need dynamically.

The reason for it to be dynamic is to avoid the unnecessary mess that can occur from having to change the different variables throughout the code.

And with Vulkan's needs being specific, especially with descriptors, having to constantly change the literals for arrays, will lead to numerous crashes, causing more delays.

After adding these descriptor sets, the pipeline was nearing completion, the last thing this pipeline needed was the semaphores and fences.

Semaphores were to enable communication between the CPU and GPU, acting as signal lights for the CPU and GPU to be given commands.

Whereas fences were more stops in the execution of the CPU while the GPU catches up, which only happened when the correct semaphore was signalled.

And only when all these elements come together, is the result a working compute pipeline in Vulkan. Leaving only one more job to do before writing the shader code.

Adding the additional descriptor set to the graphics pipeline to be input into the vertex shader, is an irritating task, this is due to having to change many different functions and literals to be implemented into the pipeline.

Luckily, the development ran into none of the potential issues that could arise and marks the beginning of writing the compute shader code and implementation of the data into the vertex shader.

Results & Evaluation

After attempting to implement the compute shader's data into the vertex shader, there were varying issues for the compute shader as well as the vertex shader.

Starting off with the compute shader, an immediate issue was the layout and dispatches, with the current method being stuck to, each time a new area was created, the respective top and bottom vertex positions were stored in vector 4s.

And this leads to lots of memory needing to be stored inside the buffer for later execution, and because this memory allocation was so large, the program would cease operations and crash, leading to no results at all.

After reducing the scale of the layout and dispatch significantly, the program was able to run, but not without some significant down time from the CPU side.

As a larger pool of models would be needed to test the efficiency of this rendering method, numerous models were put into the scene, however a

limitation of this project is that the model is repeated made, causing the program to significantly slow.

However, even with this slowness, the program does execute but not completely working as intended. Mainly due to culling not working as originally thought.

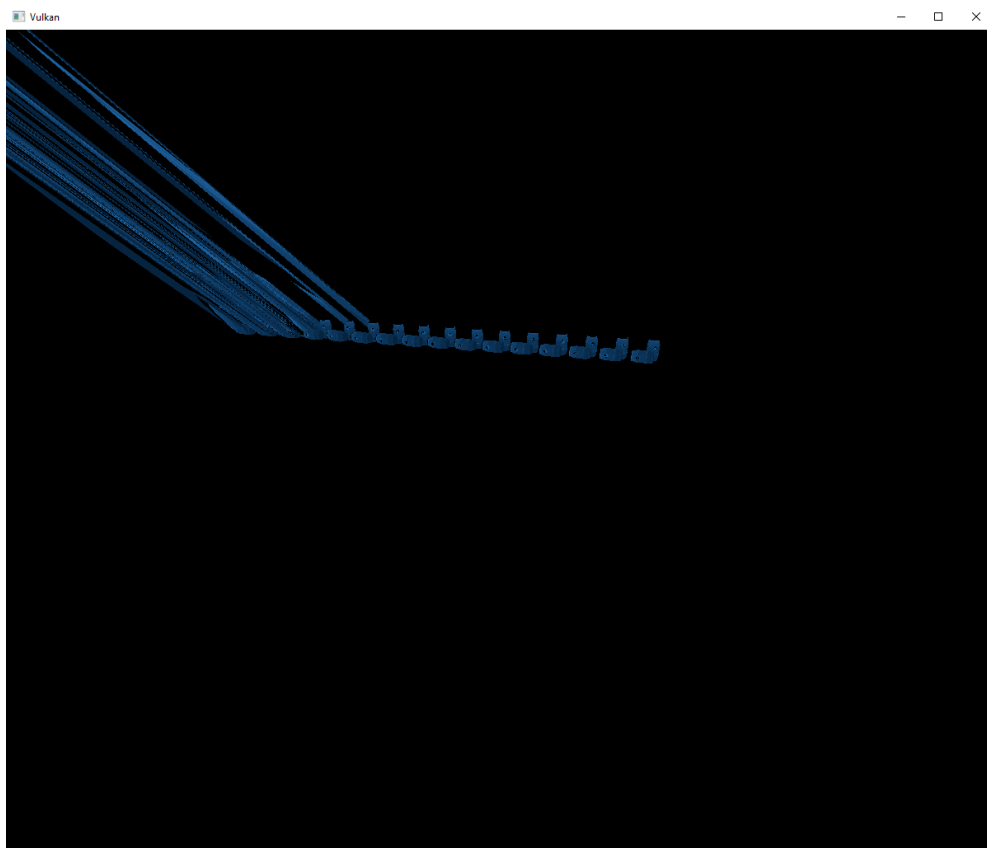


Figure 15: Output Image of Octree Grid Culling

In Figure 15, there is an obvious example of an error occurring where, instead of culling the correct vertices, the models are being stretched across the screen.

Now, this is the output that was expected to happen, the original cube surrounding all these models isn't large enough.

So, when the vertex shader run through the vertices, a vertex that can't find an area within the cube is being displaced to $[-2, -2, -2]$, a space outside of the normalised device coordinates, as seen in Figure 16.

```

int relevantCube() {
    // Runs through the amount of cubes within the grid
    for (int i = 0; i < 64; i++) {
        OctreeDim oct = grid.OOOutput[i];

        vec3 botXYZ = oct.bot.xyz;
        vec3 topXYZ = oct.top.xyz;

        // Checking to see which cube has all the correct data - slow but still works
        if (all(greaterThanEqual(inPosition, botXYZ)) &&
            all(lessThanEqual(inPosition, topXYZ))) {
            return i;
        }
    }

    return -1;
}

void main() {
    int index = relevantCube();

    OctreeDim currentGrid = grid.OOOutput[index];

    vec3 botXYZ = currentGrid.bot.xyz;
    vec3 topXYZ = currentGrid.top.xyz;

    vec4 botOctVert = ubo.proj * ubo.view * ubo.model * vec4(botXYZ, 1.0);
    vec4 topOctVert = ubo.proj * ubo.view * ubo.model * vec4(topXYZ, 1.0);

    // Example of Clip Space formulae using the vertex position
    vec3 clipSpace = vec3(gl_Position.x / gl_Position.w, gl_Position.y / gl_Position.w, gl_Position.z / gl_Position.w);

    vec3 octVert0 = vec3(botOctVert.x / botOctVert.w, botOctVert.y / botOctVert.w, botOctVert.z / botOctVert.w);
    vec3 octVert1 = vec3(topOctVert.x / topOctVert.w, topOctVert.y / topOctVert.w, topOctVert.z / topOctVert.w);

    bool isBotOctWithinClipSpace = (all(greaterThanEqual(octVert0.xyz, vec3(0.0))) && all(lessThanEqual(octVert0.xyz, vec3(1.0))));
    bool isTopOctWithinClipSpace = (all(greaterThanEqual(octVert1.xyz, vec3(0.0))) && all(lessThanEqual(octVert1.xyz, vec3(1.0))));

    if (!isBotOctWithinClipSpace || !isTopOctWithinClipSpace) {
        gl_Position = vec4(-2.0);
        return;
    }

    gl_Position = ubo.proj * ubo.view * ubo.model * vec4(inPosition, 1.0);

    fragColor = inColor;
    fragTexCoord = inTexCoord;
}

```

Figure 16: The Vertex Shader Code used

Admittedly, this is not an optimal piece of shader code, the relevant cube function uses a literal to find which cube a vertex belongs to, instead of it being passed in through a push constant or the uniform buffer.

This causes an issue for changing the number of cubes that are being compared against, so if the number of cubes changed, the developer would have to go into the shader to change the literal to the correct number.

What the Vertex Shader does

Explanation of what Figure 16 shows:

- Starting at the main function, before finding out whether the cube is within normalised device coordinates, it first goes to the relevant

cube function.

- This relevant cube function goes through all cubes generated in the compute shader and using the position input from the uniform buffer, checks whether that vertex is within any of the cubes.
- If the function can find any cube that achieves those conditions, it returns the index of which cube it is.
- However, if the function cannot find any cube that the vertex belongs to, then the return value of the function is -1.
- Then returning to the main function, using the return value, find the bottom and top vertices position in world space.
- After the transformation, checks whether those vertices are within the camera's frustum or clip space.
- And if that check returns false, the vertex position (represented as 'gl_position') is changed to [-2, -2, -2], which also occurs if the index is -1.
- Else, the vertex position is given the correct position, and is displayed correctly, like the right side of Figure 15.

What the Error could be

If the right side of the Figure 15 is being displayed with the correct values, the left side is becoming 'stretched' due to either:

1. The relevant cube function is not acquiring the cube belonging to those vertices.
2. The areas the octree division is creating aren't correct, and even though the index isn't -1, the check for NDC is returning false.

Regardless of either of these options however, due to time constraints it was regrettably left to focus on other aspects of this project.

Looking into the Compute Shader

```
struct OctreeDim {
    vec4 top;
    vec4 bot;
};

layout (binding = 0) uniform OctreeUBO{
    vec3 bottomVertPos;    // The overall 'furthest' point away (either the smallest vertex position numbers or negative numbers)
    uint padding0;
    vec3 length;           // The length of the original bounds
    uint padding1;
    int divisions;         // The amount of divisions taken
};

layout(std140, binding = 1) buffer OutputBuffer {
    OctreeDim ODOOutput[];
} grid;

// Constant Variable inside 'Constants.h' must be the same as layout
layout (local_size_x = 4, local_size_y = 4, local_size_z = 4) in;

void main() {
    // Have the invocation id scale up for the size of the box e.g. x = 0 - 7, y++, x = 0 y = 1, still the 8th cube
    uint index = gl_GlobalInvocationID.x + gl_GlobalInvocationID.y + gl_GlobalInvocationID.z;

    float cubeEdgeLengthX = length.x / divisions;
    float cubeEdgeLengthY = length.y / divisions;
    float cubeEdgeLengthZ = length.z / divisions;

    vec3 prunedCubeBotVert = vec3(
        (bottomVertPos.x + (cubeEdgeLengthX * gl_GlobalInvocationID.x)),
        (bottomVertPos.y + (cubeEdgeLengthY * gl_GlobalInvocationID.y)),
        (bottomVertPos.z + (cubeEdgeLengthZ * gl_GlobalInvocationID.z))
    );

    vec3 prunedCubeTopVert = vec3(
        prunedCubeBotVert.x + cubeEdgeLengthX,
        prunedCubeBotVert.y + cubeEdgeLengthY,
        prunedCubeBotVert.z + cubeEdgeLengthZ
    );

    grid.ODOOutput[index].top = vec4(prunedCubeTopVert, index);
    grid.ODOOutput[index].bot = vec4(prunedCubeBotVert, index);
}
```

Figure 17: The Compute Shader code used

Explanation of what Figure 17 shows:

- The 'OctreeDim' struct is the combination of an area's top and bottom vertices. Used for the output of the compute shader and input for the vertex shader.
- Next, the uniform buffer, with the data of creating the octree grid, that has 3 main elements:
 - .1. The bottom most vertex position of the cube covering the scene.
 - .2. The length of each of the cube's edges.

.3. The number of divisions that are taking place along one face of the cube.

- Past that, is the storage buffer used for transmitting the octree data to the vertex shader.
- The most important element of the shader, the layout which determines the thread group size the compute shader structures itself as, as shown in Figure 11.
- Then the main function, which every thread of the group runs.

What are some of the Project's Regrets

An immediate problem that comes to mind for both is the compilation problems that would occur, not due to misspelt variables or such.

But the chore of constantly having to go into the folder or command window to run the batch script to turn the shader from GLSL to SPIR-V.

This doesn't mean that the choice of API was wrong, it was just the fact that some time could been put into finding of method of compiling shaders to improve workflow.

Aside from that small irritation, the compute shader code was quite similar to what was worked on in the other shaders.

However, the main issue that stands out from implementing this shader, is the fact that the logic couldn't be tested properly until its full functionality was written.

This resulted in halts in the development and implementation, and because its functionality couldn't be tested, any logic changes or math done had to be tested via paper and pencil.

Which caused an even larger backlog of tasks in the timeline, and being unsure or losing the goal of the project would result in a failure of what is trying to be tested.

To summarise, a lot of indecision leading to many halts and often confusion in the development of this project.

What changes could be made to Improve the Project if it was Restarted?

If this project was to be picked up again, the most important aspect that would require the most focus would be testing the implementation of this technique against another.

Testing how fast the frame time was between the techniques, determining whether either is easier to implement than the other, and weighing the pros and cons of each in a comparable list.

Another element that would be worth to improve, is having a clear and plain understanding of the design of the compute shader, for example:

- How many cubes are being generated.
- What data needs to be input into the shader.
- How is the data being output.
- What is the process this octree is be generated.

As said previously, much more time was spent deliberating what the shader was getting and passing along that development came to a halt too many times.

And if this was going to be implemented in a dynamic scenario, it may be worth implementing bounding volume hierarchies for the dynamic objects.

One major change that would be changed is the presentation of files, simplifying down the functions and separating out the functions further to make it easier to go through the file.

Due to how limited the scope of this project became, it would be beneficial to attempt this in the mesh shader stage instead of the vertex shader stage.

Mainly to refocus on the model getting culled instead of the vertices of a model.

REFERENCES:

Halo: CE Image – Retro Review – Halo: Combat Evolved – Here We Go. (2017). Available at: <https://bagogames.com/retro-review-halo-combat-evolved-go/>.

Halo 2 Image – Probably a hot take but does anyone else think Halo 2 classic graphics are pretty bad for 2004. (2023). [Reddit] Available at: https://www.reddit.com/r/halo/comments/16ksm9i/probably_a_hot_take_but_does_anyone_else_think/?captcha=1.

Rasterisation Image / GIF - Drummyfish (2019). *Top-left triangle rasterization rule.gif*. [Wikipedia] Available at: <https://commons.wikimedia.org/w/index.php?curid=80204437>.

Raytracing in one weekend – Shirley, P., David Black, T., & Hollasch, S. (2025). *Ray Tracing in One Weekend* (Version 4.0.2) [Review of *Ray Tracing in One Weekend*]. <https://raytracing.github.io/books/RayTracingInOneWeekend.html>

NVIDIA Cyberpunk Raytracing - Burnes, A. (2023, April 11). Cyberpunk 2077: Technology Preview Of New Ray Tracing Overdrive Mode Out Now [Review of *Cyberpunk 2077: Technology Preview Of New Ray Tracing Overdrive Mode Out Now*]. *NVIDIA GeForce*. <https://www.nvidia.com/en-gb/geforce/news/cyberpunk-2077-ray-tracing-overdrive-update-launches-april-11/>

Open3D's voxelisation tutorial - Open3D (2020). Voxelization. Available at:

<https://www.open3d.org/docs/0.12.0/tutorial/geometry/voxelization.html>.

Open3D's octree tutorial - Open3D (2023). Voxelization. Available at:

<https://www.open3d.org/html/tutorial/geometry/octree.html>

Culling Image - Chrysanthou, Y. (2001). *A Survey of Visibility for Walkthrough Applications* [Review of *A Survey of Visibility for Walkthrough Applications*]. https://www.researchgate.net/figure/Three-types-of-visibility-culling-techniques-1-View-Frustum-Culling-2-back-face_fig1_2440562

Loggini, R. (2022). *Compute Shaders in D3D12* [Review of *Compute Shaders in D3D12*].

<https://logins.github.io/graphics/2020/10/31/D3D12ComputeShaders.html>

NVIDIA. (n.d.). Ray Tracing [Review of *Ray Tracing*]. *NVIDIA DEVELOPER*. <https://developer.nvidia.com/discover/ray-tracing>

Vulkan. (n.d.). Compute Shader [Review of *Compute Shader*]. *Vulkan Tutorial*. https://vulkan-tutorial.com/Compute_Shader

GeeksforGeeks. (2020, July 18). *Painter's Algorithm in Computer Graphics*. GeeksforGeeks. <https://www.geeksforgeeks.org/dsa/painters-algorithm-in-computer-graphics/>

Study for Applying Computer-generated Images to Visual Simulation. (1969).

DoomWiki.org. (n.d.). *Doom rendering engine*. [online] Available at: https://doomwiki.org/wiki/Doom_rendering_engine.